



Architectural Coverage Framework Using Reference Model

Darshak Koshiya, Sharvil Panandikar, Jiahan Zhang

Tenstorrent AI ULC
Santa Clara, California, United States

<https://tenstorrent.com/en>

ABSTRACT

With the growing complexity of the Instruction Set Architecture (ISA) extensions comes the responsibility of adding comprehensive coverage models to CPU verification methodology. In this, we explore an automation flow for using the Instruction Set Simulator (ISS) to autogenerate architectural coverage models as well as sampling. Our flow is adaptable across ISS-only simulations as well as RTL co-simulation environments.

Table of Contents

<i>Architectural Coverage Framework Using Reference Model</i>	<i>1</i>
<i>1. Introduction</i>	<i>3</i>
<i>2. Issues with traditional coverage implementation</i>	<i>3</i>
2.1 Developing coverage models for large ISA extensions.....	3
2.2 Standalone ISS simulations	3
2.3 Reuse of ISS for coverage model generation	3
<i>3. Overview.....</i>	<i>4</i>
3.1 ISS Library.....	5
3.2 Coverage model generation.....	5
3.3 Coverage model sampling.....	7
<i>4. Results</i>	<i>8</i>
<i>5. Conclusions</i>	<i>10</i>
5.1 Microarchitectural coverage:.....	10
5.2 Large width variables:.....	10
<i>6. References</i>	<i>10</i>

Table of Figures

Figure 1. Coverage Flow Overview	4
Figure 2 : Sample ISS Library API for coverage generation.....	5
Figure 3 : Architectural Coverpoint Enumeration.....	6
Figure 4: Sample autogenerated class for SATP csr.....	6
Figure 5 : Sample usage of user-defined coverpoint using autogenerated cp_pkg.....	7
Figure 6 : Sample ISS library API for sampling coverage.....	7
Figure 7: DPI interface for sending sampled values from ISS to coverage model	8
Figure 8: Sample cp_pkt structure that is used to send sampled values.....	8
Figure 9 : Sample autogenerated enumeration inside cp_pkg.sv.....	8
Figure 10 : Sample autogenerated covergroup in cp_pkg.....	9
Figure 11: Top level TB flow to receive ISS values and sample coverage model.....	9
Figure 12. Coverage report for standalone simulation.....	9

1. Introduction

CPU designs have evolved over the last few decades with the growing need for ISA improvements to support new domains like HPC, AI, etc. As the ISA extensions get complex, the design and verification effort needed to close the design gets challenging. To support the verification of complex ISA requirements, functional coverage is an important metric. In this paper, we present a flow crafted towards generating coverage collateral using information from the instruction set simulator (ISS). An ISS is a software reference model of the ISA that simulates faster than the RTL simulation because it only simulates at the ISA level (no timing/clock cycle information). Typically, the ISS and other reference models are developed and used as part of the verification effort. The flow allows user-written coverage to be assimilated into the framework so as to provide the benefits of SystemVerilog language features along with autogenerated coverage models. We also support standalone ISS simulations to gain early confidence in stimulus generators and testbench while the RTL is still under development. We have demonstrated our work using the RISC-V ISA and have established a working flow for generating coverage using an ISS [1].

2. Issues with traditional coverage implementation

To gain confidence in the stimulus and overall health of the design, functional coverage is a prime aspect. Some of the challenges for traditional coverage flows are listed as below:

2.1 Developing coverage models for large ISA extensions

Functional coverage for large ISA extensions is time consuming. To write covergroups manually can lead to manual errors and increased development time. Legacy code building over time can lead to maintenance issues. Moreover, autogenerated flows have not been able to effectively utilize SystemVerilog features to create coverage for ISA corner cases. Most proprietary coverage flows only allow users to express the coverage intent through strict file formats which are then fed to auto-generation frameworks that emit the coverage models. Users cannot add to or modify the coverage models, hindering the quality of coverage.

The flow presented in this paper provides a way to autogenerate functional coverage which allows the user to add cases as required, on top of the autogenerated coverage.

2.2 Standalone ISS simulations

It is a common observation that RTL development can overlap with verification during the early developmental stages. During the initial phase of the test planning, there can be stimulus quality issues, and it is nice to have early feedback to determine the improvements. The coverage models are written later, causing the stimulus holes only to be found during closure. Our flow helps to generate a quick coverage model over the ISA and allows for standalone ISS simulation, which can provide feedback for determining the holes at an earlier stage.

2.3 Reuse of ISS for coverage model generation

The ISS or reference model is a key requirement for any testbench environment and can be treated as a source of information, as it is compliant with the specifications. To write coverage, we can query the reference model to extract information and use it to autogenerate coverage, while also using it for functionality checks.

In this paper, we will explore the ability of using ISS for getting information about the ISA in a structured format which can be used to generate SystemVerilog packages that can be reused across various simulation environments.

3. Overview

In subsequent sections, we present a detailed insight into the coverage flow (Figure 1). The coverage flow has been demonstrated for the RISC-V ISA. Our flow generates the coverage model at build/compile time (both C++ and SystemVerilog components) and samples the coverage model at runtime. When running a test with coverage enabled, there is almost no difference from non-coverage-enabled tests, aside from additional linking done during build time (coverage model) and dynamic linking (of the sampling library) with the ISS. Coverage is collected in the normal way by the SystemVerilog simulator. The DUT can be optionally run in co-simulation with the ISS, which may be useful for coverage cases like external interrupt scenarios, which the ISS cannot model accurately on its own. The benefits of running the ISS in standalone mode (without the DUT) is the ability to collect coverage before the DUT is ready, and the generally faster performance of the ISS compared to the DUT's slow simulation times.

In our implementation, we use Whisper [2] as the ISS and Bazel [3] as our build system. Whisper is a C++ RISC-V ISS supporting the RVA23 profile [4] and provides a curated set of ISA information for both coverage generation and sampling. Bazel is a build system that allows us to seamlessly manage dependencies. It is used for managing C++/SystemVerilog compiles and tests, as well as repository/dependency management. In our presented flow, we use Bazel to link the coverage library components against Whisper and support different use contexts (for example, the standalone testbench or the co-simulated DUT) without placing a heavy burden on the user.

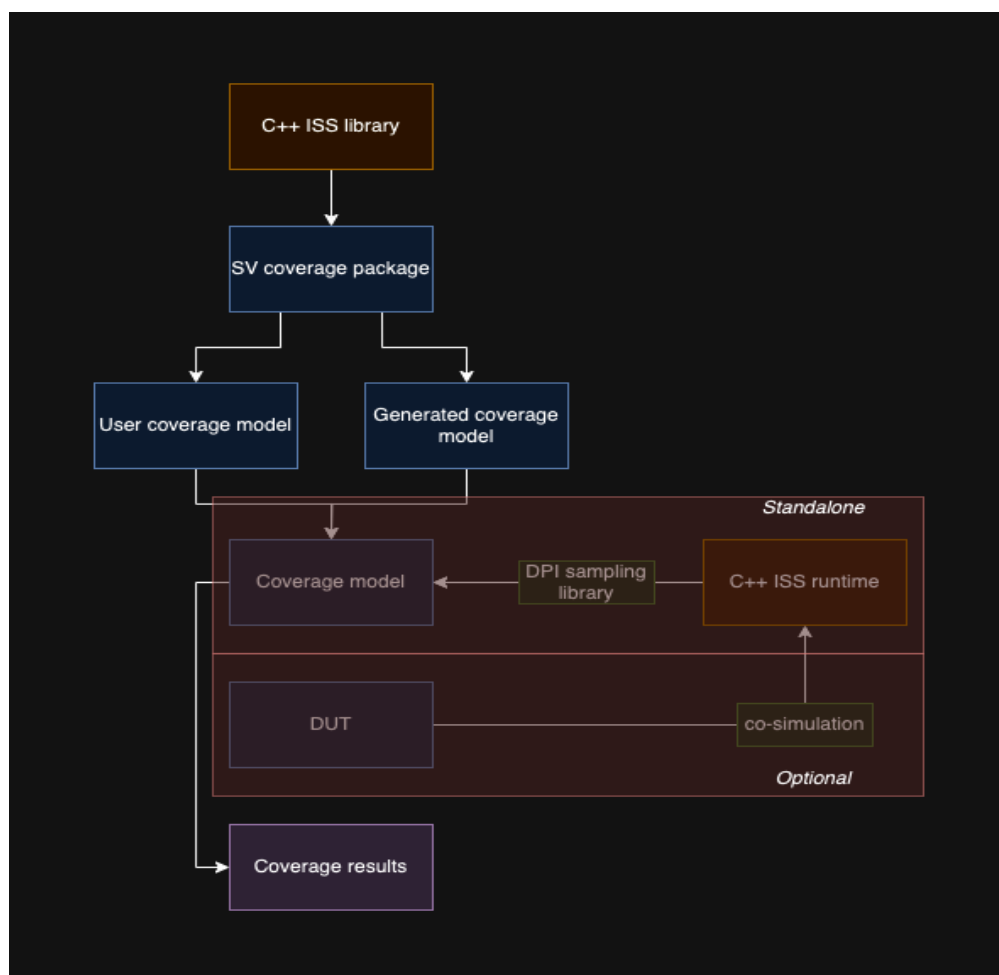


Figure 1. Coverage Flow Overview

The flow has 3 distinct parts as mentioned below:

1. ISS Library for extracting coverage specific information.
2. Coverage model generation
3. Coverage model sampling

3.1 ISS Library

The C++ ISS library is a self-contained library, which queries the ISS to capture potentially relevant ISA-level coverage information. The information is then presented as enumerations or attributes which are data structures that provide information about architectural state such as registers, instructions, etc. which can be used for coverage generation and sampling. This format is provided for ease of use of the coverage writer to effectively express desired information. The ISS is responsible for helping generate the coverage model by providing ISA-relevant information and sampling this same information during runtime. For example, the ISS exposes an API like the available page sizes, which the ISS library queries to construct page size coverage information in the form of an enum, and which will later be sampled during test runtime.

```
static constexpr const char* ptePageSize(Mode m, uint32_t level)
{
    if (m == Mode::Bare)
        return "";
    ...

    if (m == Mode::Sv32)
    {
        if (level == 1) return "4M";
    }

    else if (m == Mode::Sv39)
    {
        if (level == 1) return "2M";
        if (level == 2) return "1G";
    }

    ...
}
```

Figure 2 : Sample ISS Library API for coverage generation

3.2 Coverage model generation

The coverage generation is a translation of the SystemVerilog enums or “attributes” derived by the C++ ISS library, free variables and enums. An attribute is a SV class which expresses information about specifications which cannot be categorized into finite states. For instance, program counter (PC) values can be of interest to write coverage on indirect branches which are manifested as jump instructions to any instruction address as indicated by the operands. The PC has a wide range of values which cannot be enumerated. Hence, PC values form an attribute.

The list of available coverpoints itself is an enum, which is used by the sampling library to map incoming values from the DPI interface to relevant coverpoints. The archInfoPoints_e represents an

enum that provides a list of all the autogenerated coverpoints which can be used to classify values during runtime.

```
typedef enum {
    POINT_PRIVILEGEMODE=13,
    POINT_NEXTPRIVILEGEMODE=14,
    ....
} archInfoPoints_e;
```

Figure 3 : Architectural Coverpoint Enumeration

We also create classes for instructions and system registers (CSR) which are well-defined components of the specification. Related information for system registers and instructions along with coverpoints is encapsulated into SystemVerilog classes. Below is an example of the satp CSR, which is part of the RISC-V ISA, and contains three fields, PPN, ASID, and MODE. We provide the fields as independent class members of the `satp` class.

```
class satp_csr;
    logic[63:0] value;
    logic[43:0] ppn;
    logic[15:0] asid;
    logic[3:0] mode;

    covergroup cg_satp with function sample (logic[15:0] asid, logic[3:0] mode, logic[43:0] ppn);
        option.per_instance = 1;
        option.name = "satp";
        coverpoint asid iff (csrenum_var == CSRENUM_SATP)
        {
            bins zeroes = {0};
            bins ones = {1};
        }
        coverpoint mode iff (csrenum_var == CSRENUM_SATP)
        {
            bins valid[] = {[0:$]};
        }
    endgroup
endclass
```

Figure 4: Sample autogenerated class for SATP csr

The generation flow also provides methods that are helpful to access fields i.e., for an instruction, the field values can be a part of the opcode or General Purpose Register(GPR) information whereas for a CSR, the field values can be a configuration value that defines behavior of the CPU. All the generated code is contained in SystemVerilog packages which can be imported across other components of the environment.

Users can add more test-specific or scenario-specific coverage on top of the autogenerated code. This enables us to use the SystemVerilog language features such intersects or covergroup options along with autogenerated code.

In the example below, we use the free variable `sinval_instr` and `privilege_mode` which are free variables that are autogenerated from the ISS information and exposed to the users from the `cp_pkg` package.

```
import cp_pkg::*;

paging__invalidation : cross sinval_instr, privilege_mode;
```

Figure 5 : Sample usage of user-defined coverpoint using autogenerated `cp_pkg`

3.3 Coverage model sampling

The sampling flow requires 2 key parts:

1. ISS sampling support at specific points in the simulation
2. Receiving values collected at specific events and sampling autogenerated and user covergroups that are relevant to that event

The ISS sampling support for our use case is at instruction retirement. The C++ ISS sampling library pulls the architectural state from the ISS, and formats the data collected for coverpoints to use.

The data collected by the C++ ISS sampling library is then sent to the SystemVerilog side using DPI. The SystemVerilog sampling flow then re-formats the data according to the covergroups and calls sample methods for generating coverage. We rely on the ISS to collect relevant information at various points of instruction execution such as page table walks, memory accesses, register accesses, etc.

When the instruction is retired or suffers a trap, all collected coverage information in the ISS is provided to the sampling library. More importantly, this happens independently of whether the ISS is executing in co-simulation mode or not and functions the same whether in standalone or co-simulation mode. The following C++ snippet provides an example of how the ISS is queried to construct information that will be used for coverage generation.

```
uint64_t virtLdStAddr, physLdStAddr = ~0;
uint64_t ldStSize = tr->lastLdStAddress(virtLdStAddr, physLdStAddr);
if (ldStSize)
{
    table.addEntry(Point::VirtLdStAddr, virtLdStAddr);
    table.addEntry(Point::PhysLdStAddr, physLdStAddr);
    ...
}
```

Figure 6 : Sample ISS library API for sampling coverage

From the ISS, we obtain the virtual and physical address of a load/store instruction and append it to a lookup table structure. In this snippet, `tr` is a “TraceRecord”, which is the architectural state obtained from the ISS.

The constructed table collected by the ISS is then presented to the SystemVerilog world over DPI calls.

```
export "DPI-C" function sample_sv;

Point p = static_cast<Point>(table.entry.first);
if (p == Point::EndOfSim) {
    // Exit sim
} else {
    sample_sv(&pkt);
}
```

Figure 7: DPI interface for sending sampled values from ISS to coverage model

The `pkt` is a common C-struct shared between the ISS library and the sampling component.

```
typedef struct {
    uint64_t cp;
    uint64_t val;
} cp_pkt;
```

Figure 8: Sample `cp_pkt` structure that is used to send sampled values

Here `cp` is the integral value of the `archInfoPoints_e` enum which is used to map the relevant coverpoints. The `val` is value collected by ISS at instruction retire to be sampled by the covergroups.

4. Results

In the following section, we present some of our results. Here we use one of the enum parameters for demonstrating the generation and sampling of an autogenerated coverage artifact.

The privilege mode is an architectural coverpoint whose values are defined by the RISC-V specification. The ISS contains information regarding privilege mode and implements spec-defined behavior.

During the coverage generation phase, we query the C++ ISS library, which provides the relevant information for privilege mode parameters. This information is then used by the coverage generation framework to auto-generate a SystemVerilog equivalent enum as shown below:

Inside autogenerated SystemVerilog package:

```
typedef enum bit [63:0] {
    PRIVILEGEMODE_MACHINE=64'd3,
    PRIVILEGEMODE_RESERVED=64'd2,
    PRIVILEGEMODE_SUPERVISOR=64'd1,
    PRIVILEGEMODE_USER=64'd0
} PrivilegeMode_e;
PrivilegeMode_e privilegemode_var;
```

Figure 9 : Sample autogenerated enumeration inside `cp_pkg.sv`

The autogenerated coverage module then creates covergroups for the enum parameter.

Inside autogenerated coverage module:

```
covergroup cg_privilege_mode;
  cp_privilege_mode: coverpoint privilegemode_var;
endgroup
```

Figure 10 : Sample autogenerated covergroup in cp_pkg

Note that the user has access to the above package and can reuse the privilegemode_var variable in their own covergroups to create interesting coverage scenarios.

Inside the top level testbench, we can then invoke the sampling calls to the covergroup and pass the value that is received on an update to the privilege mode from the C++ ISS runtime library.

Inside Top Level TB:

```
privilegemode_var = PrivilegeMode_e'(<value coming from ISS runtime during sampling>);
cg_privilege_mode.sample();
```

Figure 11: Top level TB flow to receive ISS values and sample coverage model

The top level TB can be invoked in co-simulation or standalone mode using ifdef macros.

Figure(2) depicts a urg-generated report after a simulation for the privilege mode case:

Excluded/Illegal bins

NAME	COUNT	STATUS
auto_PRIVILEGEMODE_RESERVED	0	Excluded
reserved_priv	0	Excluded

Covered bins

NAME	COUNT	AT LEAST	STATUS	TEST	COUNT
auto_PRIVILEGEMODE_MACHINE	367470927	1		T1	367470
auto_PRIVILEGEMODE_SUPERVISOR	107306351	1		T1	107306
auto_PRIVILEGEMODE_USER	63602252	1		T1	636022

Figure 12. Coverage report for standalone simulation

5. Conclusions

Our coverage framework allows us to effectively develop architectural coverage independent of the state of the DUT. We obtain architectural coverage collateral well before the design is ready to run those tests, while flexibly reusing the same framework when the design is ready in co-simulation. We use user-written coverage and auto-generated coverage to reduce the burden on coverage writers. In addition, we have used this flow in practice for determining our test coverage.

With the current framework, we were able save considerable man-hours spent for developing architectural coverage.

Some of the limitations of the existing flow are as follows:

5.1 Microarchitectural coverage:

The flow cannot support microarchitectural coverage. These are implementation specific scenarios that are out of the scope of the ISA. This can be interface level coverages, cases covering RTL flows for instructions etc., which cannot be generalized across designs.

5.2 Large width variables:

Due to DPI interfaces, the simulations can slow down for large-width variables as seen with vectors, SIMD instruction operands etc. These cases also need considerable processing of the sampled values for coverage, for example, coverage involving large-width vectors would need a different DPI interface or require multiple DPI calls to be handled.

6. References

- [1] Riscv/Riscv-Isa-Manual. 2017. RISC-V, 15 Jan. 2025. GitHub, <https://github.com/riscv/riscv-isa-manual>.
- [2] Tenstorrent/Whisper. 2021. Tenstorrent AI ULC, 14 Jan. 2025. GitHub, <https://github.com/tenstorrent/whisper>.
- [3] Bazelbuild/Bazel. 2014. Bazel, 15 Jan. 2025. GitHub, <https://github.com/bazelbuild/bazel>.
- [4] Riscv/Riscv-Profiles. 2021. RISC-V, 26 Jan. 2025. GitHub, <https://github.com/riscv/riscv-profiles>.